

CHƯƠNG 10. ĐƯỜNG ĐI NGẮN NHẤT

Trong thực tế, ta thường gặp những bài toán như: Chọn một hành trình tiết kiệm nhất (về chi phí, thời gian,...) trên một mạng giao thông đường bộ, đường thủy, hay hàng không. Bài toán lập lịch thi công các công đoạn trong một công trình lớn sao cho tiết kiệm chi phí và thời gian nhất. Bài toán chọn lựa đường truyền tin tối ưu trong mạng thông tin,...v.v. Để giải quyết hiệu quả các bài toán này, người ta quy về dạng bài toán tìm đường đi ngắn nhất trong đồ thị. Trong chương này, chúng ta sẽ xem xét một số thuật toán tìm đường đi ngắn nhất thông dụng trong đồ thị.

10.1. ĐỊNH NGHĨA

Cho $G = (V, E)$ là một đồ thị có trọng số. Ký hiệu $a(u, v) \in \mathbb{R}$ là trọng số (độ dài) của cạnh (u, v) . Nếu dãy $v_0, v_1, v_2, \dots, v_p$ là một đường đi trên G thì độ dài của nó bằng tổng trọng số của các cạnh trong đường đi đó.

$$\text{Độ dài của đường đi } v_0, v_1, v_2, \dots, v_p = \sum_{i=1}^p a(v_{i-1}, v_i).$$

Giả sử có nhiều đường đi từ v_0 đến v_p , đường đi ngắn nhất là đường đi có độ dài nhỏ nhất.

Điều kiện tiên quyết cho bài toán tìm đường đi ngắn nhất giữa 2 đỉnh trong đồ thị có lời giải là đồ thị phải không có chu trình âm. Thật vậy, nếu đồ thị có chu trình âm thì ta có thể hướng đường đi vào chu trình này và sau đó có thể đi một số lần đủ lớn vòng quanh chu trình âm để được một đường đi với độ dài đủ nhỏ (!). Lúc đó sẽ không thể có đường đi ngắn nhất. Do vậy ta mặc nhiên coi tất cả các đồ thị được xem xét trong chương này đều không có chu trình âm.

10.2. THUẬT TOÁN DIJKSTRA

Thuật toán Dijkstra dùng để tìm đường đi ngắn nhất từ đỉnh s đến các đỉnh còn lại trong đồ thị. Thuật toán được áp dụng cho đồ thị có trọng số các cung không âm.

10.2.1. Ý tưởng thuật toán

- Đầu vào
 - Đồ thị có hướng $G=(V,E)$ với n đỉnh.
 - $s \in V$ là đỉnh xuất phát.
 - $a[u,v] \geq 0$ với $u,v \in V$ là ma trận trọng số
- Đầu ra
 - Khoảng cách từ s đến tất cả các đỉnh còn lại $d[v]$, $v \in V$.
 - $\text{Truoc}[v]$, $v \in V$ là đỉnh nằm trước v trong đường đi ngắn nhất từ s đến v .

Các bước thuật toán như sau:

Bước 1: Đặt nhãn cho tất cả các đỉnh trong tập $T = V \setminus \{s\}$

Bước 2: Nếu $T = \emptyset$ thì đi đến Bước 5.

Trái lại chọn $u \in T$ gần s nhất ($d(u)$: min)

Bước 3: Loại u ra khỏi T : $T = T \setminus \{u\}$; Cố định nhãn của u

Bước 4: Gán lại nhãn các đỉnh của T (theo đường đi ngắn nhất từ s qua u).

Quay lại Bước 2

Bước 5: Xuất các nhãn cố định (chính là khoảng cách ngắn nhất)

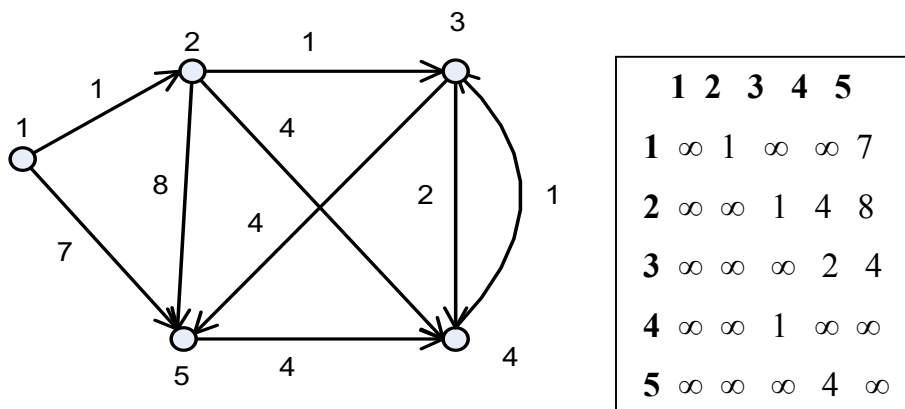
10.2.2. Cài đặt thuật toán

Sau đây là thuật toán được cài đặt bằng mã giả, người đọc có thể tham khảo mã nguồn cài đặt bằng ngôn ngữ lập trình C ở phần phụ lục của giáo trình.

```
void Dijkstra;
{
    for ( $v \in V$ ) /* Khởi tạo  $d$  và  $Truoc$  */
    {
         $d[v] = a[s,v]$ ;
         $Truoc[v] = s$ ;
    }
     $d[s] = 0$ ;
     $T = V \setminus \{s\}$ ;
    while ( $T \neq \emptyset$ )
    {
        Tìm  $u \in T$  sao cho  $d(u) = \min \{ d(z) : z \in T \}$ 
         $T = T \setminus \{u\}$ ; /* Cố định nhãn của  $u$  */
        for ( $v \in T$ ) do
            if ( $d[v] > d[u] + a[u,v]$ ) then
                {
                     $d[v] = d[u] + a[u,v]$ ;
                     $Truoc[v] = u$ ;
                }
    }
} /* Độ phức tạp của thuật toán là  $O(n^2)$  */
```

Ví dụ:

Sử dụng thuật toán Dijkstra, tìm đường đi ngắn nhất từ đỉnh 1 đến tất cả các đỉnh còn lại trong đồ thị sau:



Hình 10.1

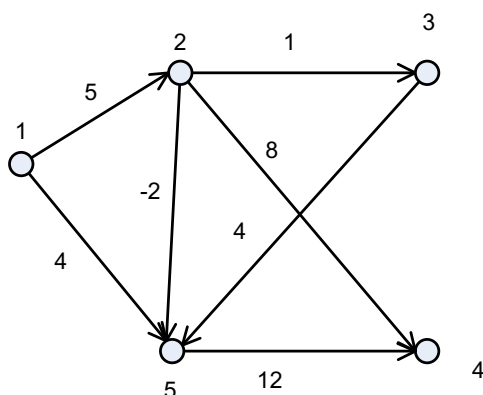
Các bước thực hiện thuật toán được minh họa qua bảng sau:

T	d[2], Truoc[2]	d[3], Truoc[3]	d[4], Truoc[4]	d[5], Truoc[5]
2, 3, 4, 5	1 , 1	∞ , 1	∞ , 1	7, 1
3, 4, 5		2 , 2	5, 2	7, 1
4, 5			4 , 3	6, 3
5				6 , 3
$\emptyset$	1, 1	2, 2	4, 3	6, 3

Từ kết quả thực hiện của giải thuật, ta có thể biết được độ dài và đường đi từ đỉnh 1 đến các đỉnh còn lại. Chẳng hạn, đường đi từ đỉnh 1 đến đỉnh 4 có độ dài là 4. Danh sách các đỉnh trên đường đi có thể biết được bằng cách truy xuất mảng Truoc[]: Truoc[4]=3, Truoc[3]=2, Truoc[2]=1. Như vậy, đường đi là: $1 \rightarrow 2 \rightarrow 3 \rightarrow 4$.

Một điểm lưu ý là, thuật toán Dijkstra không áp dụng cho đồ thị có cung trọng âm vì trong một số trường hợp, cung có trọng số âm khiến giải thuật thực hiện không cho kết quả chính xác.

Chẳng hạn đối với đồ thị dưới đây, khi áp dụng thuật toán tìm đường đi ngắn nhất từ 1 đến 5 sẽ có kết quả là 4, trong khi kết quả chính xác là: $5-2=3$.



Hình 10.2

• **Nhận xét:**

Thuật toán Dijkstra được đánh giá là thực hiện nhanh, tốn ít chi phí tính toán. Lý do: vì sau mỗi bước lặp của giải thuật, ta sẽ nhận được kết quả đường đi ngắn nhất đến một đỉnh nào đó. Đỉnh này sẽ được loại bỏ, không tham gia tính toán trong các bước sau. Chính vì thế, số phép tính toán giảm dần theo thời gian. Tuy nhiên, đó cũng là nguyên nhân khiến thuật toán có khả năng cho kết quả sai khi đồ thị có cung trọng số âm như trong ví dụ hình 7-2. Một giải thuật khác cũng cho phép tìm đường đi ngắn nhất từ 1 đỉnh đến các đỉnh còn lại của đồ thị nhưng cho phép đồ thị có cung trọng số âm là thuật toán Ford-Bellman.

10.3. THUẬT TOÁN FORD-BELLMAN

Thuật toán Ford-Bellman dùng để tìm đường đi ngắn nhất từ một đỉnh s đến tất cả các đỉnh còn lại của đồ thị.

10.3.1. Ý tưởng thuật toán

▪ Đầu vào:

- Đồ thị có hướng $G=(V,E)$ với n đỉnh.
- $s \in V$ là đỉnh xuất phát.
- $a[u,v]$ với $u,v \in V$ là ma trận trọng số

▪ Đầu ra:

- Khoảng cách từ s đến tất cả các đỉnh còn lại $d[v]$, $\forall v \in V$.
- $Truoc[v]$, $\forall v \in V$ là đỉnh nằm trước v trong đường đi ngắn nhất từ s đến v

Cho đồ thị có hướng, có trọng số $G = (V, E)$. Trọng số các cung của G được tính như sau:

- $TrongSo(u, v) = \infty$ nếu $(u, v) \notin E$.
- $TrongSo(u, v) = a(u, v)$ nếu cung $(u, v) \in E$.

Thuật toán tìm đường đi ngắn nhất $d(v)$ từ đỉnh s , $\forall v \in V$:

+ Xét $u \in V$. Nếu $d(u) + TrongSo(u, v) < d(v)$ thì ta thay $d(v) = d(u) + TrongSo(u, v)$.

+ Quá trình này sẽ được lặp lại với tất cả các đỉnh u có thể, với tất cả các đỉnh v có thể cho đến khi không thể làm tốt hơn các giá trị $d(v)$.

10.3.2. Cài đặt thuật toán

Sau đây ta cài đặt thuật toán bằng mã giả.

```
void Ford_Bellman()
{
    for ( $v \in V$ ) /* Khởi tạo  $d$  và  $Truoc$  */
    {
```

```

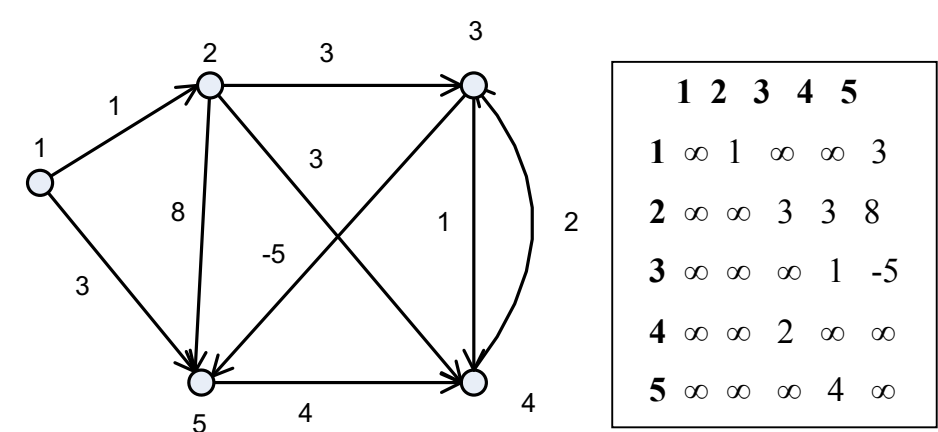
        d[v] = a[s,v];
        Truoc[v] = s;
    }
    d[s] = 0;
    for (k = 1; k < n-1; k++) /* Thực hiện n-2 lần */
    /**/ for (v ∈ V \ {s})
        for (u ∈ V)
            if (d[v] > d[u] + a[u,v])
            {
                d[v] = d[u] + a[u,v];
                Truoc[v] = u;
            }
    } /* Độ phức tạp của thuật toán là O(n3) */

```

Thuật toán trên sẽ thực hiện vòng lặp **/**/** n-2 lần. Trong thực tế số lần lặp có thể ít hơn và thuật toán có thể dừng lại ngay nếu các giá trị d[v] không thay đổi nữa.

Ví dụ:

Áp dụng thuật toán Ford-Bellman, tìm đường đi ngắn nhất từ đỉnh 1 đến tất cả các đỉnh còn lại của đồ thị sau:



Hình 10.3

Các bước chạy thuật toán được thể hiện trong bảng sau. Thuật toán DỪNG khi hai dòng kết quả cuối cùng giống nhau (khi đó giá trị d[v] của các đỉnh không được cải tiến nữa).

k	d[5], Truoc[5]	d[4], Truoc[4]	d[3], Truoc[3]	d[2], Truoc[2]
1	3, 1	∞, 1	∞, 1	1, 1
2	3, 1	4, 2	4, 2	1, 1
3	-1, 3	4, 2	4, 2	1, 1

4	-1, 3	3, 5	4, 2	1, 1
5	-1, 3	3, 5	4, 2	1, 1

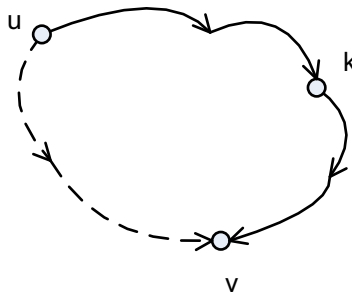
10.4. THUẬT TOÁN FLOYD

Thuật toán Floyd cho phép tìm đường đi ngắn nhất giữa tất cả các cặp đỉnh trong đồ thị.

10.4.1. Ý tưởng thuật toán

Giả sử k là một đỉnh nào đó của đồ thị. Xét các cặp đỉnh u, v . Ta sẽ tìm đường đi ngắn nhất từ u đến v mà có đi qua k theo công thức:

$$a(u, v) = \min(a(u, v), a(u, k) + a(k, v))$$



Hình 10.4

Nếu đường đi hiện có từ u đến v dài hơn đường đi từ u đến k cộng với đường đi từ k đến v thì đường đi từ u đến v hiện tại sẽ được thay thế bởi đường đi mới qua đỉnh k . Đồng thời ta phải cập nhật thông tin lưu vết của các đường đi.

10.4.2. Cài đặt thuật toán

▪ Đầu vào

- Đồ thị cho bởi ma trận trọng số: $a[i, j]$, $i, j = \overline{1..n}$

▪ Đầu ra: Hai ma trận

- Ma trận đường đi ngắn nhất giữa các cặp đỉnh:

$d[i, j]$, $i, j = \overline{1..n}$

$d[i, j]$ là độ dài đường đi ngắn nhất từ i đến j

- Ma trận ghi nhận đường đi.

$p[i, j]$, $i, j = \overline{1..n}$

$p[i, j]$ là đỉnh nằm trước đỉnh j trong đường đi ngắn nhất từ i đến j .

```
void Floyd;
{
    for (i = 1; i <= n; i++) /* Khởi tạo */
        for (j = 1; j <= n; j++)
        {
            d[i, j] = a[i, j];
```

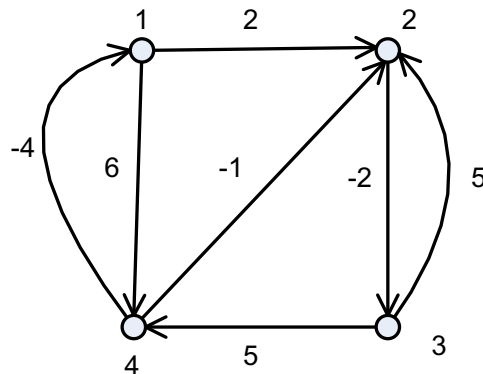
```

p[i,j] = i;
    }
    for (k = 1; k <= n; k++) /* 3 vòng lặp */
        for (i = 1; i <= n; i++)
            for (j = 1; j <= n; j++)
                if (d[i,j] > d[i,k] + d[k,j])
                {
                    d[i,j] = d[i,k] + d[k,j];
                    p[i,j] = p[k,j];
                }
    }

```

Ví dụ:

Áp dụng thuật toán Floyd, tìm đường đi ngắn nhất giữa tất cả các cặp đỉnh trong đồ thị sau:



Hình 10.5

- Khởi tạo hai ma trận:

D: Ma trận lưu giá trị đường đi. ∞

P: Ma trận lưu vết.

	1	2	3	4
1	∞	2	∞	6
2	∞	∞	-2	∞
3	∞	5	∞	5
4	-4	-1	∞	∞

	1	2	3	4
1	1	1	1	1
2	2	2	2	2
3	3	3	3	3
4	4	4	4	4

▪ Kết quả các bước lặp của thuật toán:

k=1

D₁

	1	2	3	4
1	∞	2	∞	6
2	∞	∞	-2	∞
3	∞	5	∞	5
4	-4	-2	∞	2

P₁

	1	2	3	4
1	1	1	1	1
2	2	2	2	2
3	3	3	3	3
4	4	4	1	4

k=2

D₂

	1	2	3	4
1	∞	2	0	6
2	∞	∞	-2	∞
3	∞	5	3	5
4	-4	-2	-4	2

P₂

	1	2	3	4
1	1	1	2	1
2	2	2	2	2
3	3	3	2	3
4	4	4	1	2

k=3

D₃

	1	2	3	4
1	∞	2	0	5
2	∞	3	-2	3
3	∞	5	3	5
4	-4	-2	-4	1

P₃

	1	2	3	4
1	1	1	2	3
2	2	3	2	3
3	3	3	2	3
4	4	4	1	2

k=4

D₄

	1	2	3	4
1	1	2	0	5
2	-1	1	-2	3
3	1	3	1	5
4	-4	-2	-4	1

P₄

	1	2	3	4
1	4	1	2	3
2	4	1	2	3
3	4	1	2	3
4	4	1	2	3

Kết quả cuối cùng của thuật toán được lưu trong hai ma trận D và P. Độ dài đường đi ngắn nhất từ đỉnh s đến đỉnh t sẽ là: D[s,t]. Đường đi cụ thể sẽ được tính từ ma trận lưu vết P.

Ví dụ: Tìm đường đi ngắn nhất từ đỉnh 3 đến đỉnh 2:

Độ dài đường đi: $D[3, 2] = 3$

$P[3, 2] = 1$. Đỉnh 1 là đỉnh trước của đỉnh 2. $P[3, 1] = 4$. Đỉnh 4 trước đỉnh 1

$P[3, 4] = 3$. Đỉnh 3 trước đỉnh 4.

Vậy đường đi ngắn nhất từ đỉnh 3 đến đỉnh 2 là: $3 \rightarrow 4 \rightarrow 1 \rightarrow 2$. Với độ dài = 3.

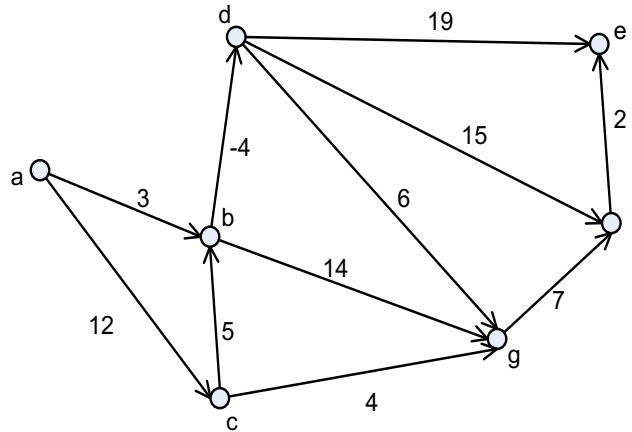
Một số kết quả có thể suy ra từ các ma trận D và P :

Nếu trên đường chéo chính của ma trận D có giá trị nào đó khác ∞ thì đồ thị sẽ có chu trình

Nếu tất cả các giá trị của D (không kể đường chéo chính) đều khác ∞ thì đồ thị liên thông mạnh

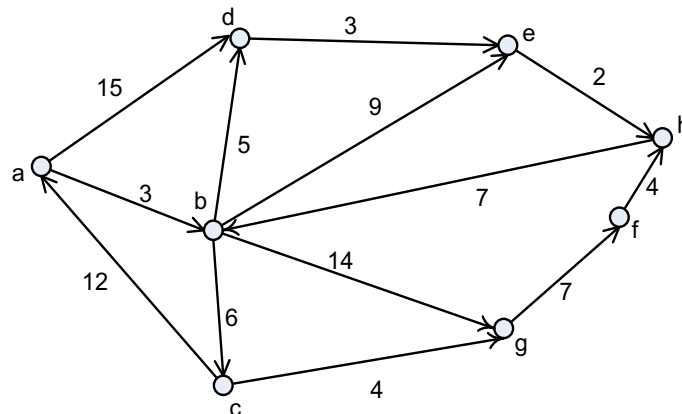
10.5. BÀI TẬP CHƯƠNG 10

1. Áp dụng thuật toán Ford-Bellman, tìm đường đi ngắn nhất từ đỉnh a đến tất cả các đỉnh trong đồ thị sau:



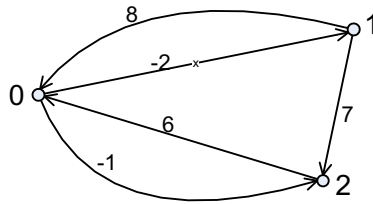
Hình 10.6

2. Áp dụng thuật toán Dijkstra, tìm đường đi ngắn nhất từ đỉnh a đến tất cả các đỉnh trong đồ thị sau



Hình 10.7

3. Áp dụng thuật toán Floyd, tìm đường đi ngắn nhất giữa các cặp đỉnh trong đồ thị sau

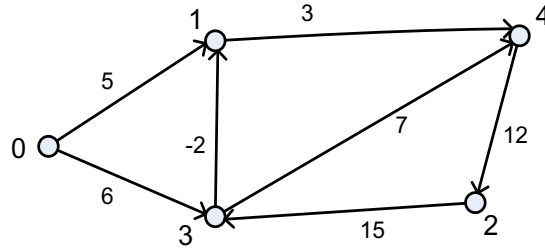


Hình 10.8

4. Viết chương trình cài đặt các thuật toán sau trên ngôn ngữ C++:

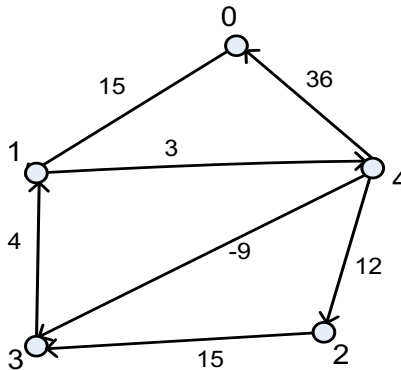
- a) Thuật toán Ford-Bellman.
- b) Thuật toán Dijkstra.
- c) Thuật toán Floyd.

5. Có thể áp dụng giải thuật Dijkstra để tìm đường đi ngắn nhất từ đỉnh 0 đến các đỉnh còn lại cho đồ thị sau được không? Giải thích lý do.



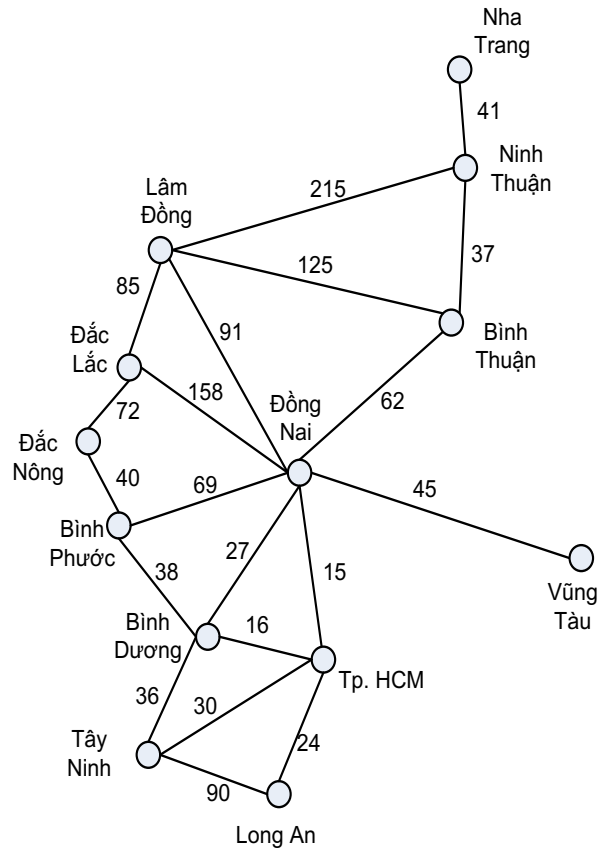
Hình 10.9

6. Minh họa từng bước quá trình tìm đường đi ngắn nhất từ đỉnh 0 đến các đỉnh còn lại của đồ thị. Giải thích lý do vì sao không tìm được đường đi ngắn nhất.



Hình 10.10

7. Cho sơ đồ các thành phố và chi phí (tính theo đơn vị nghìn đồng) để di chuyển (bằng ô tô) giữa các thành phố như sau:



Hình 10.11

Giả sử một công ty du lịch đặt tại TP. Hồ Chí Minh muốn tổ chức các tour du lịch từ TP. Hồ Chí Minh đến các tỉnh, thành phố khác. Hãy chỉ đường cho công ty này sao cho các tour là ít tốn kém về tiền vé nhất.